

1.ARRAY INDEX OUT OF BOUNDS EXCEPTION

■ Basic :Information Experiment No 04

Date :17/09/2026

Title :Implementation of ArrayIndexOutOfBoundsException using Exception Handling

Objective / Aim :

To write a Java program that demonstrates exception handling using try-catch blocks and handles an invalid array index using ArrayIndexOutOfBoundsException.



System Requirements

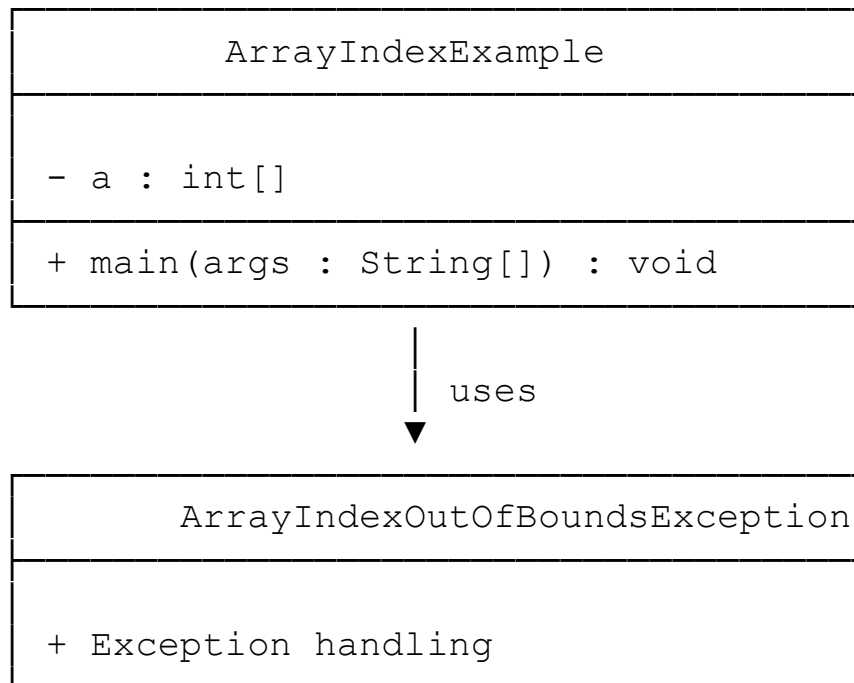
- Hardware: Standard PC/Laptop, Core i3/i5 Processor, 4GB/8GB RAM.
- Software: Windows/Linux/macOS, Java Development Kit (JDK 17 or JDK 21), IntelliJ IDEA / Eclipse / VS Code / NetBeans.



Algorithm / Logic Description

1. Start the program.
2. Create an integer array `a` with the values `10, 20, 30`.
3. Place the array access statement inside the `try` block.
4. Try to access the element at index `5`.
5. Since the array contains only three elements, index `5` is invalid.
6. An `ArrayIndexOutOfBoundsException` is raised.
7. The `catch` block catches the exception.
8. Display the message `"Array index is out of bounds"`.
9. Stop the program.

UML Class Diagram



Source Code

```
package exceptionhandling;

class ArrayIndexExample {
    public static void main(String[] args) {

        try {
            int a[] = {10, 20, 30};

            System.out.println(a[5]);
        }
        catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("Array index is out of bounds");
        }
    }
}
```

Compilation & Execution Commands

File name: `ArrayIndexExample.java`

Compile the code:

```
javac -d . ArrayIndexExample.java
```

Run the compiled bytecode:

```
java exceptionhandling.ArrayIndexExample
```

Output Screen

```
"C:\Program Files\Java\jdk-24\bin"
Array index is out of bounds

Process finished with exit code 0
```

2.CLASS NOT FOUND EXCEPTION

Basic Information

- Experiment No: 05
- Date: 17/09/2026
- Title: Implementation of ClassNotFoundException using Exception Handling

Objective / Aim

To write a Java program that demonstrates exception handling using `try-catch` blocks and handles a missing class using `ClassNotFoundException`.



System Requirements

- Hardware: Standard PC setup (Core i3/i5 Processor, 4GB/8GB RAM).



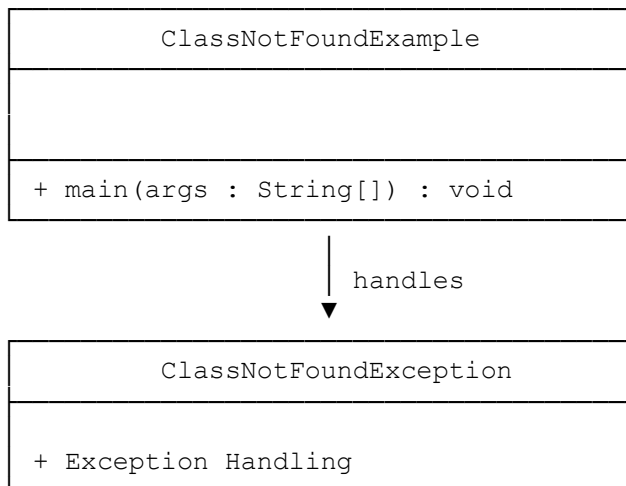
- Software: Windows, Java Development Kit (JDK 17), IntelliJ IDEA

Algorithm / Logic Description

1. Start the program.
2. Use the `try` block to load a class using `Class.forName()`.
3. Pass "Student123" as the class name.
4. Since the specified class is not available, a `ClassNotFoundException` occurs.
5. The `catch` block catches the exception.
6. Display the message "Class not found".
7. Stop the program.



UML Class Diagram



Source Code

```
package exceptionhandling;

class ClassNotFoundExample {
    public static void main(String[] args) {
```

```
try {  
    Class.forName("Student123");  
}  
catch (ClassNotFoundException e) {  
    System.out.println("Class not found");  
}  
}
```

Compilation & Execution Commands

Save the file as:

ClassNotFoundException.java

Compile the code:

```
javac -d . ClassNotFoundException.java
```

Run the compiled bytecode:

```
java exceptionhandling.ClassNotFoundException
```

Output Screen

```
"C:\Program Files\Java\jdk-24\bin\j  
Class not found  
  
Process finished with exit code 0
```

3. ILLEGAL STATE EXCEPTION

Basic Information

- Experiment No: 06
- Date: 17/09/2026

- Title: Implementation of `IllegalStateException` using Exception Handling

Objective / Aim

To write a Java program that demonstrates exception handling using `try-catch` blocks and handles an invalid state of an object using `IllegalStateException`.



System Requirements

- Hardware: Standard PC setup (Core i3/i5 Processor, 4GB/8GB RAM).
- Software: Windows/Linux/macOS, Java Development Kit (JDK 17 or JDK 21), IntelliJ IDEA / Eclipse / NetBeans / VS Code.

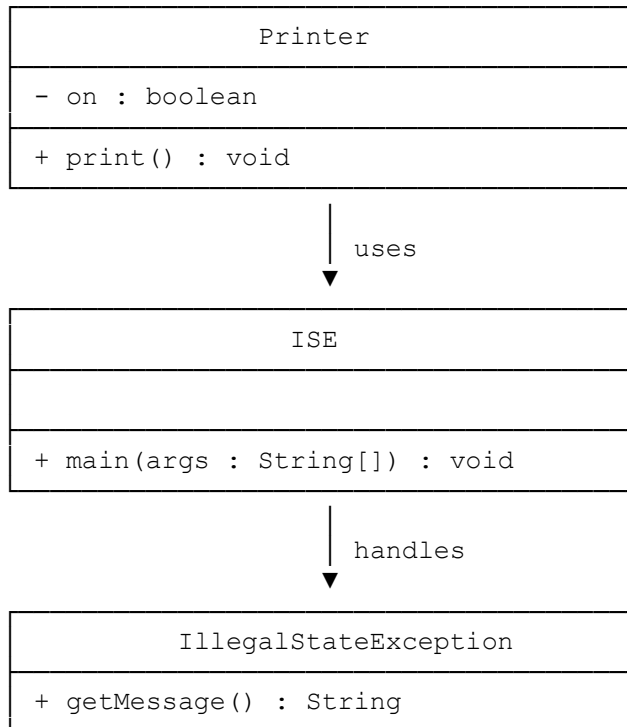


Algorithm / Logic Description

1. Start the program.
2. Create a class named `Printer`.
3. Declare a boolean variable `on` and initialize it to `false`.
4. Define a `print()` method inside the `Printer` class.
5. Check whether the printer is turned off using `if (!on)`.
6. If the printer is off, throw an `IllegalStateException` with the message `"Printer is turned off"`.
7. Create an object of the `Printer` class in the `main()` method.
8. Call the `print()` method inside the `try` block.
9. Catch the `IllegalStateException` using the `catch` block.
10. Display the exception message using `e.getMessage()`.
11. Stop the program.



UML Class Diagram



Source Code

```
package exceptionhandling;

class Printer{
    boolean on = false;
    void print() {
        if (!on) {
            throw new IllegalStateException("Printer is turned off");
        }
        System.out.println("Printing...");
    }
}

public class ISE {
    public static void main(String[] args) {
        Printer p = new Printer();
        try {
            p.print();
        }
        catch (IllegalStateException e) {
            System.out.println(e.getMessage());
        }
    }
}
```

Compilation & Execution Commands

Save the file as:

```
ISE.java
```

Compile the code:

```
javac -d . ISE.java
```

Run the compiled bytecode:

```
java exceptionhandling.ISE
```



Output Screen

```
"C:\Program Files\Java\jdk-24\bin\java  
Printer is turned off
```

```
Process finished with exit code 0  
|
```

4. ILLEGAL ARGUMENT EXCEPTION



Basic Information

- Experiment No: 11
- Date: 17/09/2026
- Title: Implementation of IllegalArgumentException using Exception Handling

Objective / Aim

To write a Java program that demonstrates exception handling using `try-catch` blocks and handles an invalid argument passed to a method using `IllegalArgumentException`.



System Requirements

- Hardware: Standard PC setup (Core i3/i5 Processor, 4GB/8GB RAM).
- Software: Windows/Linux/macOS, Java Development Kit (JDK 17 or JDK 21), IntelliJ IDEA / Eclipse / NetBeans / VS Code.

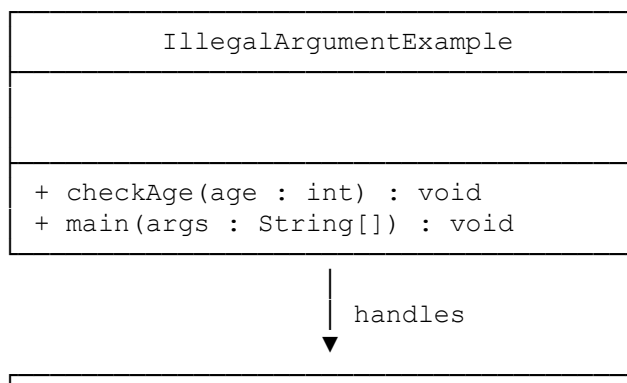


Algorithm / Logic Description

1. Start the program.
2. Create a class named `IllegalArgumentExample`.
3. Define a method `checkAge(int age)` to check the given age.
4. Check whether the age is less than zero.
5. If the age is negative, throw an `IllegalArgumentException` with the message "Age cannot be negative".
6. If the age is valid, display the age.
7. Create an object of the `IllegalArgumentExample` class in the `main()` method.
8. Call the `checkAge()` method with the value `-23` inside the `try` block.
9. Catch the `IllegalArgumentException` using the `catch` block.
10. Display the exception message using `e.getMessage()`.
11. Stop the program.



UML Class Diagram



IllegalArgumentException
+ getMessage() : String

Source Code

```
package exceptionhandling;

class IllegalArgumentException {

    void checkAge(int age) {

        if (age < 0) {
            throw new IllegalArgumentException("Age cannot be negative");
        }

        System.out.println("Age is: " + age);
    }

    public static void main(String[] args) {
        IllegalArgumentException il=new IllegalArgumentException();
        try {
            il.checkAge(-23);
        }
        catch (IllegalArgumentException e) {
            System.out.println(e.getMessage());
        }
    }
}
```

Compilation & Execution Commands

Save the file as:

```
IllegalArgumentException.java
```

Compile the code:

```
javac -d . IllegalArgumentException.java
```

Run the compiled bytecode:

```
java exceptionhandling.IllegalArgumentException
```



Output Screen

```
"C:\Program Files\Java\jdk-24\bin\jav
```

```
Age cannot be negative
```

```
Process finished with exit code 0
```

5.NEGATIVE ARRAY EXCEPTION



Basic Information

- Experiment No: 07
- Date: 17/09/2026
- Title: Implementation of NegativeArraySizeException using Exception Handling



Objective / Aim

To write a Java program that demonstrates exception handling using `try-catch` blocks and handles the creation of an array with a negative size using `NegativeArraySizeException`.



System Requirements

- Hardware: Standard PC setup (Core i3/i5 Processor, 4GB/8GB RAM).
- Software: Windows, Java Development Kit (JDK 17), IntelliJ IDEA

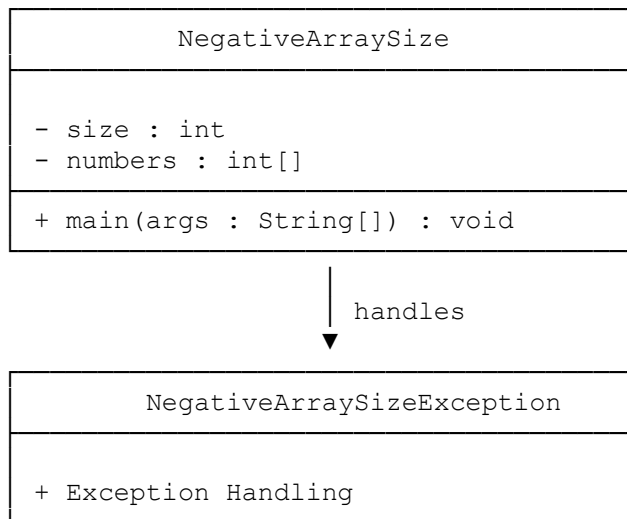


Algorithm / Logic Description

1. Start the program.

2. Declare an integer variable `size` and initialize it to `-2`.
3. Place the array creation statement inside the `try` block.
4. Try to create an integer array using the negative size.
5. Since the array size cannot be negative, a `NegativeArraySizeException` occurs.
6. The `catch` block catches the exception.
7. Display the message "Array size cannot be negative".
8. Stop the program.

UML Class Diagram



Source Code

```
package exceptionhandling;

public class NegativeArraySize {
    public static void main(String[] args) {
        try {
            //size cannot be negative it has to be either 0 or 1,2,-----
            int size = -5;
```

```

        int[] numbers = new int[size];
        System.out.println("Array created");
    }
    catch (NegativeArraySizeException e) {
        System.out.println("Array size cannot be negative");
    }
}
}

```

Compilation & Execution Commands

Save the file as:

NegativeArraySize.java

Compile the code:

```
javac -d . NegativeArraySize.java
```

Run the compiled bytecode:

```
java exceptionhandling.NegativeArraySize
```



Output Screen

```
"C:\Program Files\Java\jdk-24\bin\java
```

```
Array size cannot be negative
```

```
Process finished with exit code 0
```

6.NULL POINTER EXCEPTION



Basic Information

- Experiment No: 08
- Date: 17/09/2026
- Title: Implementation of NullPointerException using Exception Handling

Objective / Aim

To write a Java program that demonstrates exception handling using `try-catch` blocks and handles accessing a method on a null reference using `NullPointerException`.



System Requirements

- Hardware: Standard PC setup (Core i3/i5 Processor, 4GB/8GB RAM).
- Software: Windows/Linux/macOS, Java Development Kit (JDK 17 or JDK 21), IntelliJ IDEA / Eclipse / NetBeans / VS Code.

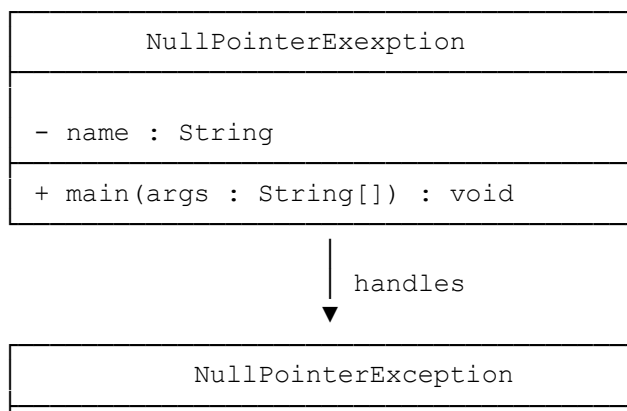


Algorithm / Logic Description

1. Start the program.
2. Declare a String variable `name` and initialize it to `null`.
3. Place the string length operation inside the `try` block.
4. Try to access the length of the string using `name.length()`.
5. Since `name` is null, a `NullPointerException` occurs.
6. The `catch` block catches the exception.
7. Display the message "The string is null. Cannot access its length."
8. Stop the program.



UML Class Diagram





Source Code

```
package exceptionhandling;

public class NullPointerException {
    public static void main(String[] args) {

        String name =null;

        try {
            System.out.println("Length of name: " + name.length());

        } catch (NullPointerException npe) {
            System.out.println("The string is null. Cannot access its length.");
        }

    }
}
```



Compilation & Execution Commands

Save the file as:

```
NullPointerException.java
```

Compile the code:

```
javac -d . NullPointerException.java
```

Run the compiled bytecode:

```
java exceptionhandling.NullPointerException
```



Output Screen

```
"C:\Program Files\Java\jdk-24\bin\java.exe" "-
The string is null. Cannot access its length.

Process finished with exit code 0
```

7.NUMBER FORMAT EXCEPTION



Basic Information

- Experiment No: 09
- Date: 17/09/2026
- Title: Implementation of NumberFormatException using Exception Handling



Objective / Aim

To write a Java program that demonstrates exception handling using `try-catch` blocks and handles invalid numeric input using `NumberFormatException`.



System Requirements

- Hardware: Standard PC setup (Core i3/i5 Processor, 4GB/8GB RAM).
- Software: Windows/Linux/macOS, Java Development Kit (JDK 17 or JDK 21), IntelliJ IDEA / Eclipse / NetBeans / VS Code.



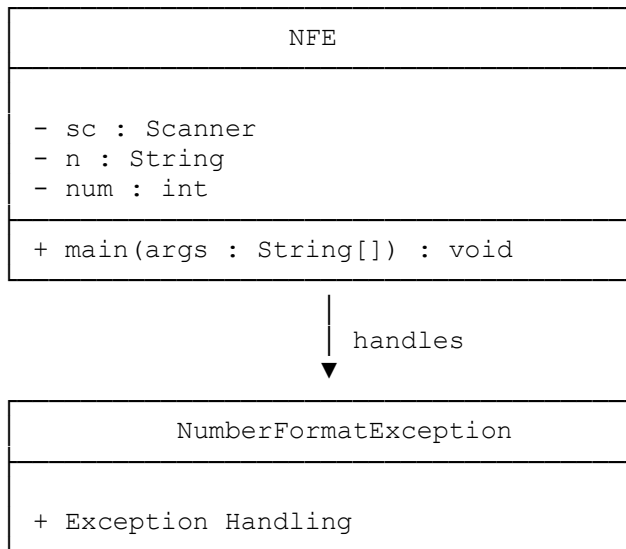
Algorithm / Logic Description

1. Start the program.
2. Create a `Scanner` object to read input from the user.
3. Display the message "Enter a number:".
4. Read the input as a `String` using `nextLine()`.
5. Place the number conversion inside the `try` block.
6. Convert the `String` into an integer using `Integer.parseInt(n)`.
7. If the input is not a valid integer, a `NumberFormatException` occurs.
8. The `catch` block catches the exception.

9. Display the message "Please enter a valid number."
10. If the input is valid, display the number.
11. Stop the program.



UML Class Diagram



Source Code

```
package exceptionhandling;

import java.util.Scanner;

public class NFE {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter a number:");
        String n = sc.nextLine();

        try {
            int num = Integer.parseInt(n);
            System.out.println("The number is: " + num);
        } catch (NumberFormatException nfe) {
            System.out.println("Please enter a valid number.");
        }
    }
}
```

Compilation & Execution Commands

Save the file as:

```
NFE.java
```

Compile the code:

```
javac -d . NFE.java
```

Run the compiled bytecode:

```
java exceptionhandling.NFE
```



Output Screen

```
C:\Program Files\Java\jdk-24\bin>java
Enter a number:
3423@#4453
Please enter a valid number.

Process finished with exit code 0
```

8.STRING INDEX OUT OF BOUNDS EXCEPTION



Basic Information

- Experiment No: 10
- Date: 17/09/2026
- Title: Implementation of StringIndexOutOfBoundsException using Exception Handling

Objective / Aim

To write a Java program that demonstrates exception handling using `try-catch` blocks and handles an invalid string index using `StringIndexOutOfBoundsException`.



System Requirements

- Hardware: Standard PC setup (Core i3/i5 Processor, 4GB/8GB RAM).
- Software: Windows/Linux/macOS, Java Development Kit (JDK 17 or JDK 21), IntelliJ IDEA / Eclipse / NetBeans / VS Code.

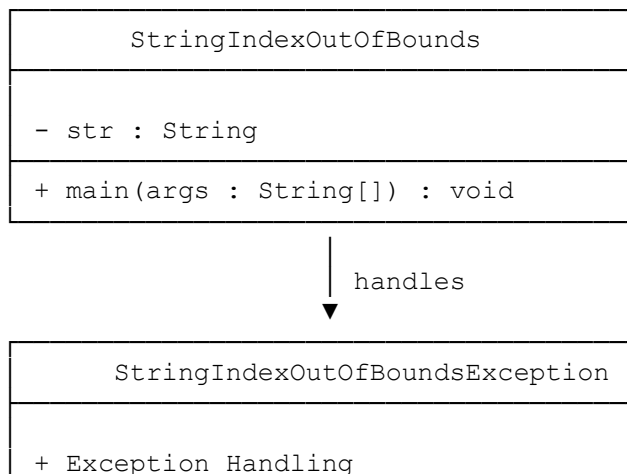


Algorithm / Logic Description

1. Start the program.
2. Create a String variable `str` and initialize it with "Java".
3. Place the string access statement inside the `try` block.
4. Try to access the character at index 10 using `charAt(10)`.
5. Since the string contains only four characters, index 10 is invalid.
6. A `StringIndexOutOfBoundsException` occurs.
7. The `catch` block catches the exception.
8. Display the message "String index is out of bounds".
9. Stop the program.



UML Class Diagram





Source Code

```
package exceptionhandling;

class StringIndexOutOfBoundsException {
    public static void main(String[] args) {

        try {
            String str = "Java";

            System.out.println(str.charAt(10));
        }
        catch (StringIndexOutOfBoundsException e) {
            System.out.println("String index is out of bounds");
        }
    }
}
```



Compilation & Execution Commands

Save the file as:

```
StringIndexOutOfBoundsException.java
```

Compile the code:

```
javac -d . StringIndexOutOfBoundsException.java
```

Run the compiled bytecode:

```
java exceptionhandling.StringIndexOutOfBoundsException
```



Output Screen

```
"C:\Program Files\Java\jdk-24\bin\j
String index is out of bounds

Process finished with exit code 0
```

9.CLASS CAST EXCEPTION



Basic Information

- Experiment No: 12
- Date: 17/09/2026
- Title: Implementation of ClassCastException using Exception Handling



Objective / Aim

To write a Java program that demonstrates exception handling using `try-catch` blocks and handles invalid type casting using `ClassCastException`.



System Requirements

- Hardware: Standard PC setup (Core i3/i5 Processor, 4GB/8GB RAM).
- Software: Windows/Linux/macOS, Java Development Kit (JDK 17 or JDK 21), IntelliJ IDEA / Eclipse / NetBeans / VS Code.

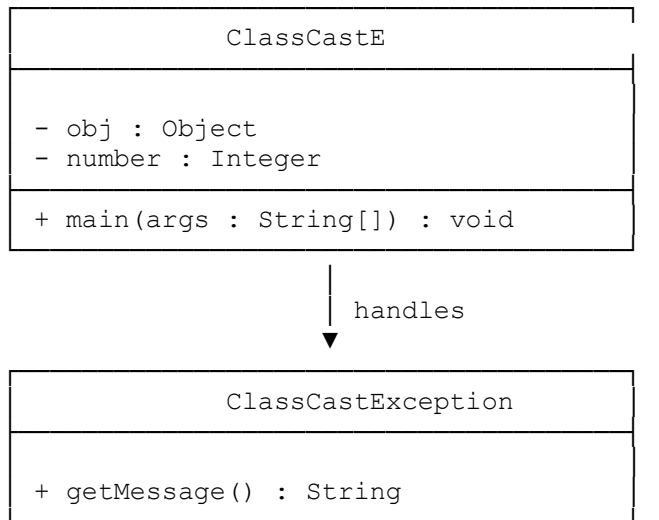


Algorithm / Logic Description

1. Start the program.
2. Create an object of the `String` class and store it in an `Object` reference.
3. Place the type casting statement inside the `try` block.
4. Try to cast the `String` object into an `Integer` object.
5. Since a `String` object cannot be cast into an `Integer` object, a `ClassCastException` occurs.
6. The `catch` block catches the exception.
7. Display the exception message using `cce.getMessage()`.
8. Stop the program.



UML Class Diagram



Source Code

```
package exceptionhandling;

// classcast exception
public class ClassCastE {
    public static void main(String[] args) {
        Object obj = new String("manodhara");
        try {
            Integer number = (Integer) obj;
        } catch (ClassCastException cce) {
            // TODO: handle exception
            System.out.println("we catch an exception called
cce"+ cce.getMessage());
        }
    }
}
```



Compilation & Execution Commands

Save the file as:

ClassCast.java

Compile the code:

```
javac -d . ClassCast.java
```

Run the compiled bytecode:

```
java exceptionhandling.ClassCast
```



Output Screen

we catch an exception called cceclass java.lang.String cannot be cast to class java.lang.Integer (java.lang.String and java.lang.Integer are in module java.base of loader 'bootstrap')

10.ARRAY STORE EXCEPTION



Basic Information

- Experiment No: 13
- Date: 17/09/2026
- Title: Implementation of ArrayStoreException using Exception Handling

Objective / Aim

To write a Java program that demonstrates exception handling using `try-catch-finally` blocks and handles storing an incompatible data type in an array using `ArrayStoreException`.



System Requirements

- Hardware: Standard PC setup (Core i3/i5 Processor, 4GB/8GB RAM).
- Software: Windows/Linux/macOS, Java Development Kit (JDK 17 or JDK 21), IntelliJ IDEA / Eclipse / NetBeans / VS Code.



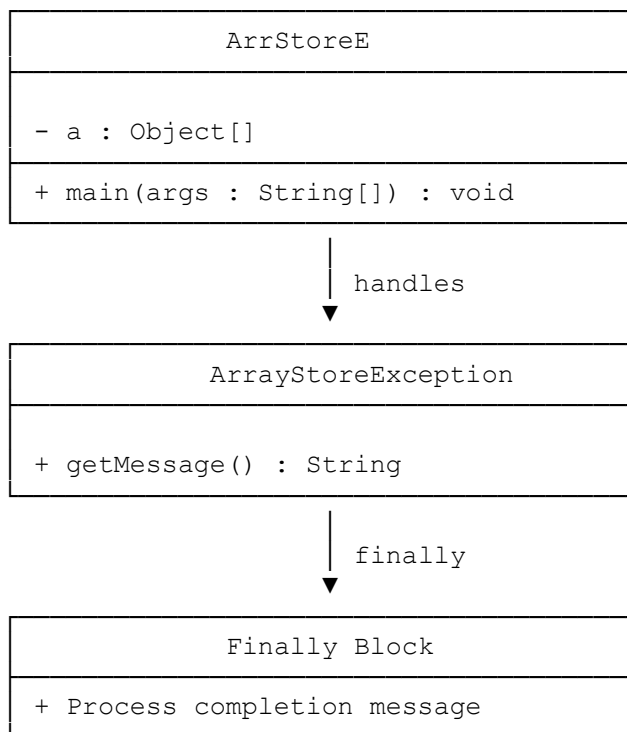
Algorithm / Logic Description

1. Start the program.
2. Create an Object array `a` that refers to a String array of size 3.
3. Place the array assignment statement inside the `try` block.

4. Try to store an Integer value 9 in the String array.
5. Since an Integer cannot be stored in a String array, an `ArrayStoreException` occurs.
6. The `catch` block catches the exception.
7. Display the message "caught is ase" along with the exception message.
8. Execute the `finally` block.
9. Display the message "the process is doned".
10. Stop the program.



UML Class Diagram



Source Code

```
package exceptionhandling;

// arraystoreexception
public class ArrStoreE {
    public static void main(String[] args) {
```

```
Object a[] = new String[3];
try {
    a[0] = Integer.valueOf(9);
} catch (ArrayStoreException ase) {
    // TODO: handle exception
System.out.println("caught is ase" +ase.getMessage());

}
finally{
    System.out.println("the process is doned");
}
}
```

Compilation & Execution Commands

Save the file as:

ArrStoreE.java

Compile the code:

```
javac -d . ArrStoreE.java
```

Run the compiled bytecode:

```
java exceptionhandling.ArrStoreE
```



Output Screen

```
caught is asejava.lang.Integer
the process is doned
```